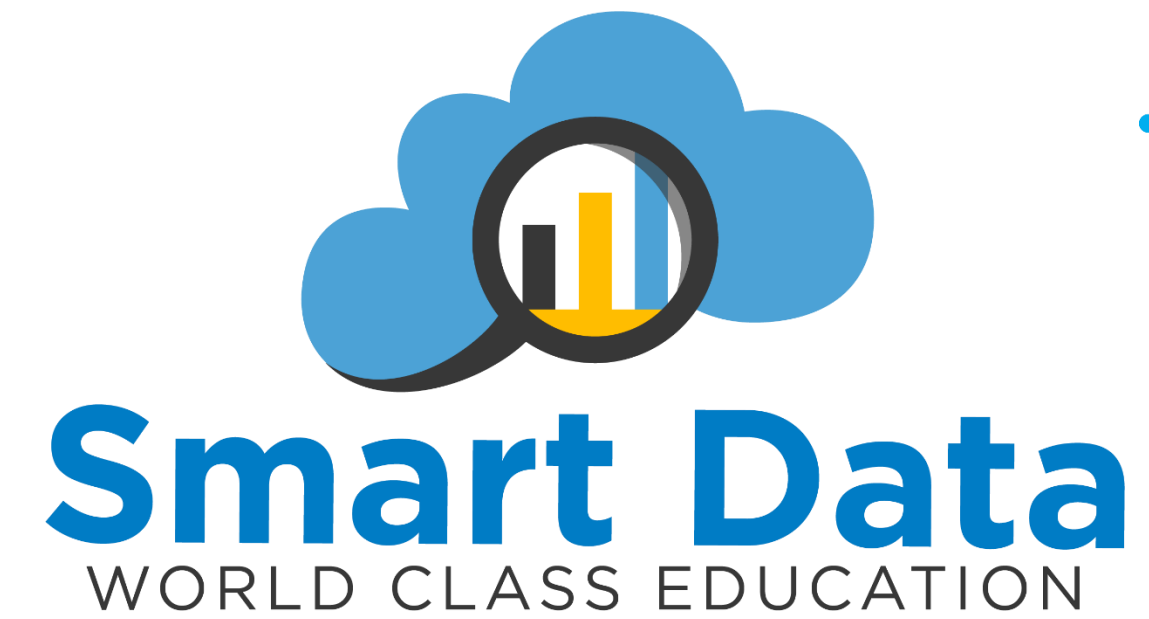
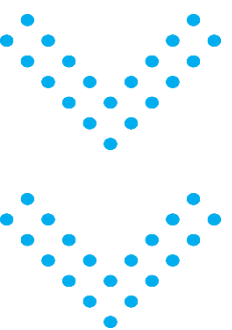
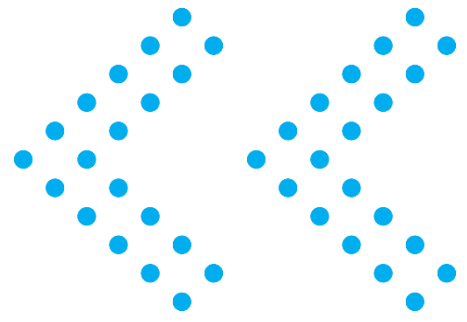
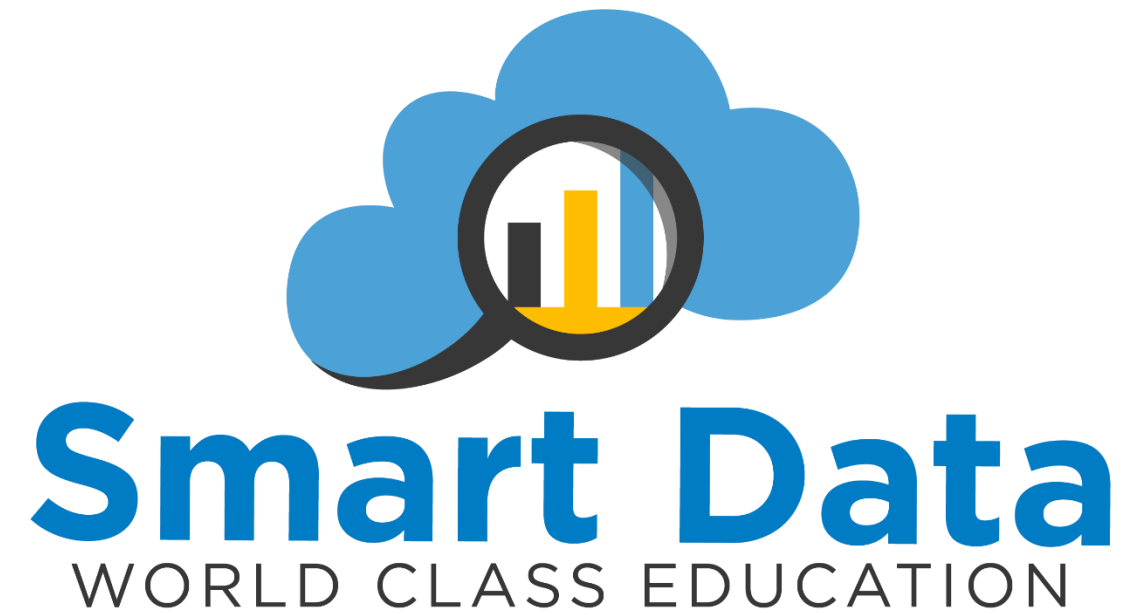




MLOps: Del Modelo al entorno Productivo



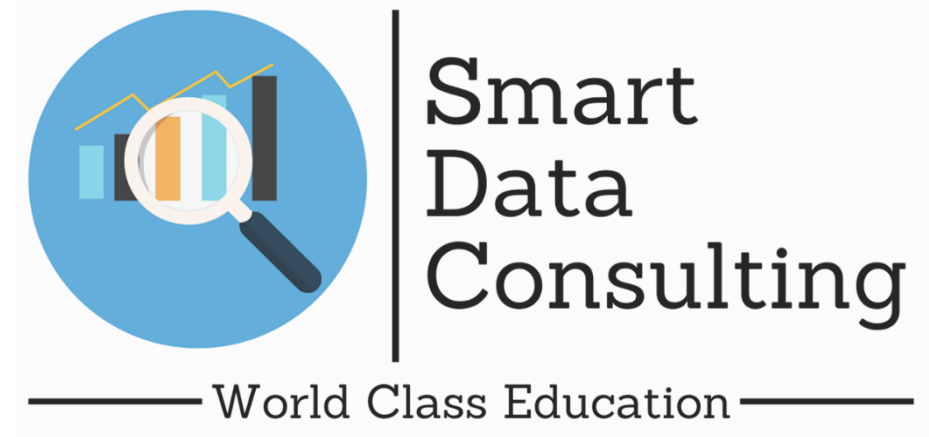


Integración y Proyecto Final

Instructor:

Julio César Bernal Fernández

Agenda General



01 Revisión del Flujo MLOps

02 Checklist de Buenas Prácticas

03 Documentación y Empaquetado

04 Presentación del Caso Aplicado

¿Qué es MLOps?

MLOps = ML + DevOps + DataOps

Conjunto de prácticas para estandarizar y optimizar el ciclo de vida de los modelos de machine learning en producción.

Automatización

CI/CD para modelos: entrenamiento, evaluación y despliegue continuo sin intervención manual.

Reproducibilidad

Versionado de código, datos y modelos para garantizar resultados consistentes en cada ejecución.

Monitoreo

Detección de drift de datos y degradación de métricas en tiempo real durante la inferencia.

Ciclo de Vida del Modelo ML



Ciclo iterativo y continuo — cada etapa retroalimenta a las anteriores

Pipeline MLOps Implementado

1

**Source
Control**

Git + DVC
Versionado de
código y datos

2

CI Pipeline

GitHub Actions
Tests unitarios,
linting

3

**Train &
Evaluar**

MLflow Tracking
Experimentos y
métricas

4

**Model
Registry**

MLflow Registry
Staging →
Production

5

Serving

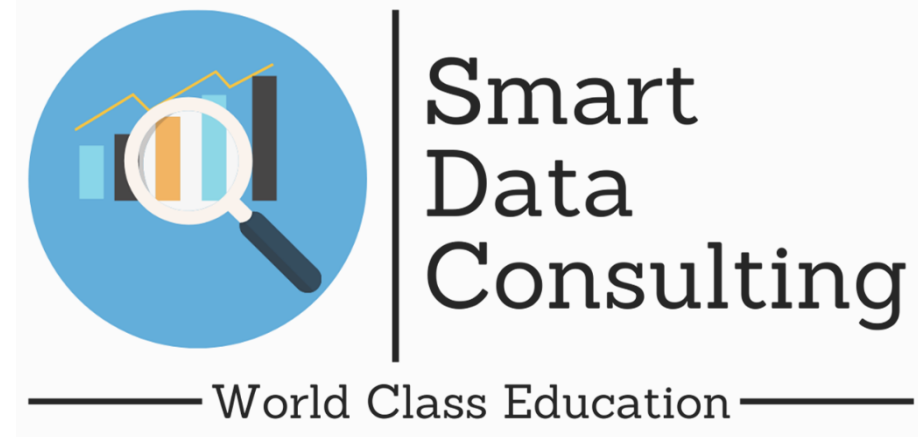
FastAPI / Seldon
REST / gRPC
endpoint

6

Monitor

Evidently AI
Drift + alertas

Stack de Herramientas MLOps



Versionado

Git, DVC, LakeFS

Código, datos y artefactos versionados con trazabilidad completa.

Orquestación

Airflow, Prefect, Kubeflow

DAGs de flujo para entrenamiento y re-entrenamiento automático.

Serving

FastAPI, BentoML, Seldon

Endpoints REST/gRPC con versionado de modelos y rollback.

Experimentación

MLflow, W&B

Registro de runs, parámetros, métricas y artefactos por experimento.

Empaquetado

Docker, Conda, Poetry

Entornos reproducibles y portables para todos los servicios.

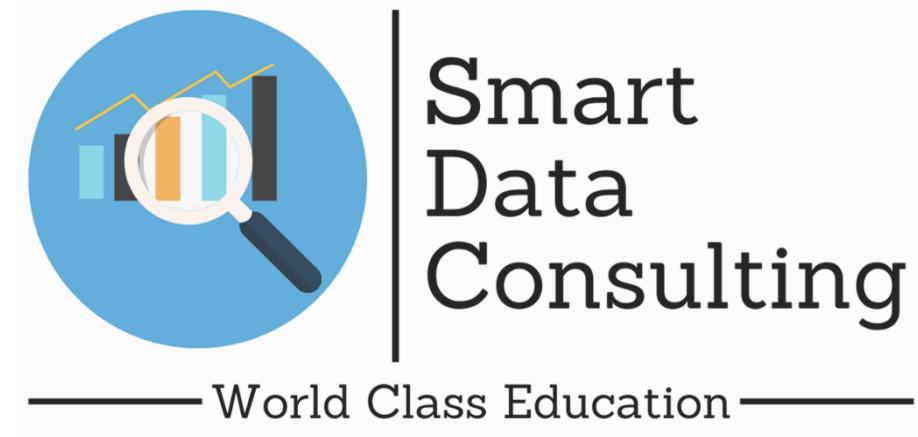
Monitoreo

Evidently, Prometheus, Grafana

Dashboard de métricas y alertas de degradación del modelo.

Versionado y Registro de Modelos

MLflow Model Registry — Flujo de Etapas



None
(experimento)

Staging
(validación)

Production
(servicio activo)

Archived
(deprecado)

- Cada modelo recibe un nombre, versión y etapa clara.
- Transiciones controladas por CI/CD y aprobaciones.
- Tags de métricas: accuracy, F1, latencia P99.
- Rollback automático si métricas caen bajo umbral.
- Integración con Serving: carga el modelo 'Production'.

Buenas Prácticas: Código y Datos



Control de versiones

Todo el código en Git con branching strategy (GitFlow o trunk-based).



DVC para datos

Datasets y modelos versionados con DVC apuntando a S3/GCS/Azure Blob.



Pruebas unitarias

Tests para funciones de transformación, preprocesamiento y validación.



Linting & formateo

Black, Flake8, isort integrados en pre-commit hooks y CI.



Schemas de datos

Validación con Great Expectations o Pandera en cada ingesta.

Buenas Prácticas: Entrenamiento



Reproducibilidad

Seeds fijos, logging de hiperparámetros, snapshots de datos de entrenamiento.



Tracking de experimentos

Cada run registrado: métricas, parámetros, artefactos y duración.



Validación cruzada rigurosa

K-Fold estratificado, evaluación temporal para series de tiempo.



Baseline comparativo

Todo modelo nuevo comparado con un baseline simple (media, regresión lineal).



Feature store

Features calculadas una vez, reutilizadas en entrenamiento e inferencia.

Buenas Prácticas: Despliegue

Contenedores Docker

Imagen reproducible con dependencias fijadas en requirements.txt o pyproject.toml.

CI/CD automatizado

Pipeline que valida, construye y despliega el modelo al pasar pruebas definidas.

API documentada

Swagger/OpenAPI con contratos claros: input schema, output schema, ejemplos.

Blue/Green deploy

Dos entornos paralelos permiten rollback instantáneo sin downtime.

Health checks

Endpoint /health y /metrics expuestos para balanceadores y orquestadores.

Seguridad

Auth, rate limiting, validación de inputs para prevenir ataques de inferencia.

Buenas Prácticas: Monitoreo en Producción



Sin monitoreo, el modelo en producción es una caja negra.

Data Drift

Detectar cambios en la distribución de las variables de entrada. Herramienta: Evidently AI, NannyML.

Concept Drift

El target cambia con el tiempo aunque los inputs sean similares. Requiere re-etiquetado periódico.

Model Performance

Accuracy, F1, AUC monitoreados con ventana temporal deslizante en datos de producción.

Latencia & Throughput

P50, P95, P99 de tiempos de respuesta. SLA definido antes del despliegue.

Alertas y Estrategias de Re-entrenamiento

¿Cuándo re-entrenar el modelo?

Por umbral de métrica

Si $\text{accuracy} < \text{umbral definido}$ o $\text{drift score} > 0.15$, disparar pipeline de re-entrenamiento.

Por calendario

Re-entrenamiento periódico (semanal, mensual) con datos frescos aunque no haya drift evidente.

Por volumen de datos

Al acumular N nuevas observaciones etiquetadas, iniciar ciclo de fine-tuning.

Re-entrenamiento continuo

Online learning: actualización incremental con cada batch de producción (casos seleccionados).

Estructura del Repositorio del Proyecto

```
my-ml-project/
├── data/
│   ├── raw/           # datos originales
│   └── processed/     # datos limpios
├── notebooks/         # EDA y prototipado
├── src/
│   ├── features/      # ingeniería de features
│   ├── models/        # entrenamiento
│   └── api/            # serving FastAPI
├── tests/              # pytest
├── configs/            # YAML de hyperparams
├── Dockerfile
├── dvc.yaml            # pipeline DVC
├── params.yaml         # parámetros reproducibles
└── README.md
```

Cookiecutter

Plantilla estándar para proyectos ML.
Genera estructura automáticamente.

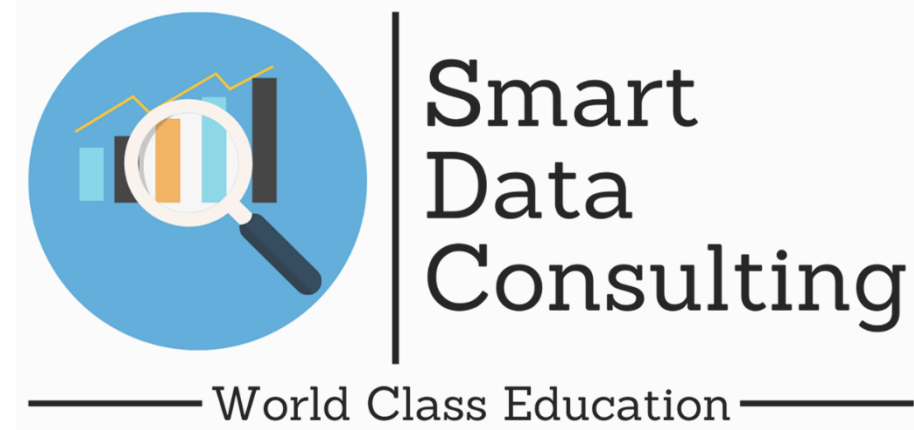
configs/

Separar hiperparámetros del código permite reproducibilidad y Optuna/hyperopt.

dvc.yaml

Define el DAG completo: cada etapa con sus inputs, outputs y comandos.

README y Documentación Técnica



Project Overview

Objetivo del negocio, problema ML, métricas de éxito y audiencia.

Metodología

Decisiones de modelado, features seleccionadas, algoritmos probados y descartados.

Uso en Producción

Cómo invocar la API, ejemplos de request/response, límites del modelo.

Setup & Instalación

Comandos exactos para reproducir el entorno: conda env, pip install, DVC pull.

Resultados

Tabla comparativa de modelos con métricas: train, validation, test. Gráficas incluidas.

Re-entrenamiento

Cuándo y cómo re-entrenar, datos mínimos requeridos, proceso de aprobación.

Empaquetado con Docker & MLflow

Dockerfile (multi-stage)

```
FROM python:3.11-slim AS base
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

FROM base AS production
COPY src/ ./src/
COPY configs/ ./configs/
COPY models/ ./models/
EXPOSE 8000
CMD ["uvicorn", "src.api.main:app",
    "--host", "0.0.0.0", "--port", "8000"]
```

MLflow Models

```
mlflow models serve -m
models:/MiModelo/Production -p 8000
```

BentoML Pack

bentoml build → bentoml containerize →
docker push

Model Card

Metadatos: autor, fecha, datos entrenamiento,
limitaciones, sesgos.

Testing del Modelo antes de Producción

Unit Tests

Funciones de feature engineering y preprocesamiento verificadas de forma aislada.

Integration Tests

Pipeline completo: input → features → modelo → output verificado end-to-end.

Performance Tests

Latencia < SLA definido. Throughput en carga peak (stress testing con Locust).

Bias & Fairness

Evaluación por subgrupos: género, región, etc. Herramientas: Fairlearn, AI Fairness 360.

Shadow Mode

Modelo nuevo recibe tráfico real pero sin afectar respuestas. Comparación offline.

A/B Testing

Fracción del tráfico al modelo nuevo. Análisis estadístico de mejora antes de rollout.

Caso Aplicado: Predicción de Churn de Clientes

Problema de Negocio

Predecir qué clientes abandonarán el servicio en los próximos 30 días para intervenir proactivamente.

Datos de Entrada

- Historial de transacciones (24 meses)
- Logs de interacción web/app
- Datos demográficos anonimizados
- Tickets de soporte al cliente
- NPS y encuestas de satisfacción

Variable Target

- Churn = 1: cliente perdido en 30d
- Churn = 0: cliente activo
- Desbalance: 85% / 15%
- Resuelto con SMOTE + class_weight
- Umbral óptimo por curva PR

Métricas Clave

- Recall en clase positiva ≥ 0.80
- Precision ≥ 0.65 (costo campaña)
- AUC-ROC ≥ 0.88
- F1-score balance precision/recall
- Lift y Gini para ranking

Feature Engineering del Caso Churn

Variables Temporales

Recencia de última compra, frecuencia mensual, tendencia de uso (creciente/decreciente).

Engagement Score

Ratio de sesiones/mes, páginas por sesión, uso de features premium del producto.

Cambios en patrón

Detección de caídas abruptas en uso ($>30\%$ vs mes anterior): señal temprana de churn.

RFM Score

Recency, Frequency, Monetary combinados en un score compuesto de riesgo de churn.

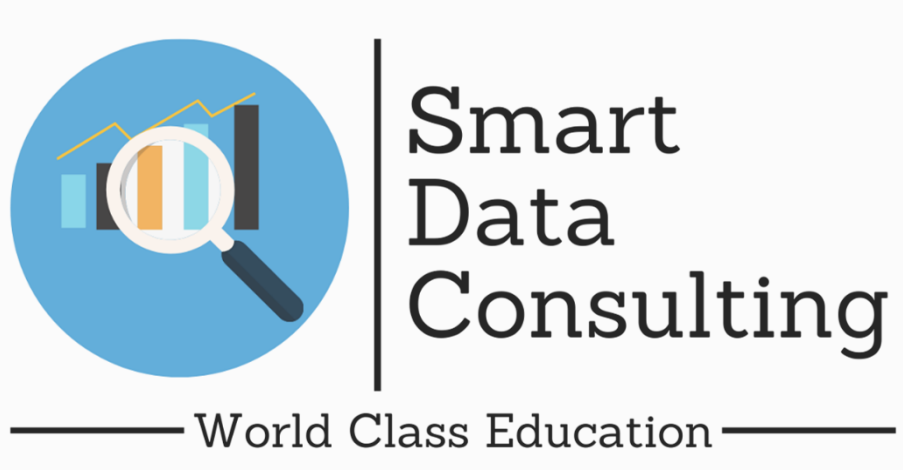
Soporte al cliente

Número de tickets, tiempo de resolución promedio, categorías de quejas frecuentes.

Interacciones

Producto cruzado entre recencia y engagement: usuarios inactivos con alto gasto = riesgo alto.

Resultados: Comparativa de Modelos



Modelo	AUC-ROC	F1	Recall	Precision	Tiempo (s)
Baseline (Reg. Log.)	0.812	0.71	0.76	0.67	1.2
Random Forest	0.871	0.78	0.82	0.74	18.4
XGBoost ★	0.903	0.83	0.86	0.80	12.1
LightGBM	0.898	0.82	0.85	0.79	6.8
Ensamble (Voting)	0.911	0.84	0.87	0.81	32.0

★ XGBoost seleccionado como modelo final por balance entre rendimiento y latencia de inferencia.

Validación: Stratified K-Fold (k=5) + evaluación temporal hold-out últimos 3 meses.

Arquitectura del Sistema en Producción



Lecciones Aprendidas y Próximos Pasos

💡 Los datos importan más que el algoritmo

El 70% del tiempo fue en limpieza y feature engineering. Un modelo simple con buenos datos supera al mejor modelo con datos sucios.

⚡ Automatizar desde el inicio

Configurar CI/CD y MLflow desde el día 1 ahorró semanas de trabajo manual al final del proyecto.

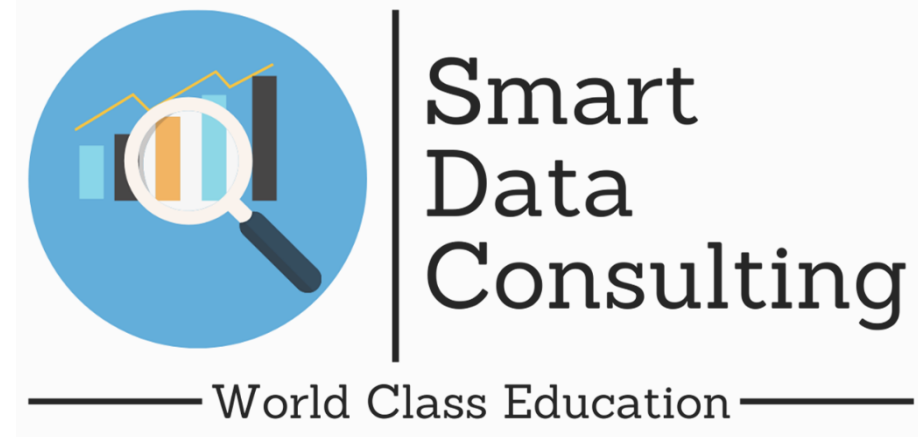
📉 El drift llega antes de lo esperado

A los 6 meses el modelo degradó un 8%. El monitoreo activo permitió detectarlo antes de impacto al negocio.

🤝 Colaboración ML + Negocio es clave

Definir el umbral de clasificación con el equipo de negocio (no con F1 solo) duplicó el ROI del modelo.

Referencias y Recursos



MLOps Principles

ml-ops.org/content/mlops-principles

DVC - Data Version Control

dvc.org/doc

Google Cloud MLOps

cloud.google.com/architecture/mlops-continuous-delivery

Feast Feature Store

feast.dev/docs

Great Expectations

greatexpectations.io/docs

BentoML - Model Packaging

docs.bentoml.com

MLflow Documentation

mlflow.org/docs/latest/index.html

Evidently AI - Monitoring

evidentlyai.com/docs

Microsoft Azure MLOps

docs.microsoft.com/azure/machine-learning/concept-model-management

FastAPI for ML serving

fastapi.tiangolo.com

Fairlearn - AI Fairness

fairlearn.org

Hidden Technical Debt (Sculley et al.)

papers.nips.cc/paper/2015/file/86df7dcfd896fcf2674f757a2463eba